

Keras API Project Exercise

The Data

I will be using a subset of the LendingClub DataSet obtained from Kaggle:

<https://www.kaggle.com/wordsforthewise/lending-club>

(<https://www.kaggle.com/wordsforthewise/lending-club>)

LendingClub is a US peer-to-peer lending company, headquartered in San Francisco, California.[3] It was the first peer-to-peer lender to register its offerings as securities with the Securities and Exchange Commission (SEC), and to offer loan trading on a secondary market. LendingClub is the world's largest peer-to-peer lending platform.

Our Goal

Given historical data on loans given out with information on whether or not the borrower defaulted (charge-off), can I build a model that can predict whether or not a borrower will pay back their loan? This way in the future when we get a new potential customer we can assess whether or not they are likely to pay back the loan. Keep in mind classification metrics when evaluating the performance of your model!

The "loan_status" column contains our label.

Data Overview

There are many LendingClub data sets on Kaggle. Here is the information on this particular data set:

	LoanStatNew	Description
0	loan_amnt	The listed amount of the loan applied for by the borrower. If at some point in time, the credit department reduces the loan amount, then it will be reflected in this value.
1	term	The number of payments on the loan. Values are in months and can be either 36 or 60.
2	int_rate	Interest Rate on the loan
3	installment	The monthly payment owed by the borrower if the loan originates.
4	grade	LC assigned loan grade
5	sub_grade	LC assigned loan subgrade
6	emp_title	The job title supplied by the Borrower when applying for the loan.*
7	emp_length	Employment length in years. Possible values are between 0 and 10 where 0 means less than one year and 10 means ten or more years.

	LoanStatNew	Description
8	home_ownership	The home ownership status provided by the borrower during registration or obtained from the credit report. Our values are: RENT, OWN, MORTGAGE, OTHER
9	annual_inc	The self-reported annual income provided by the borrower during registration.
10	verification_status	Indicates if income was verified by LC, not verified, or if the income source was verified
11	issue_d	The month which the loan was funded
12	loan_status	Current status of the loan
13	purpose	A category provided by the borrower for the loan request.
14	title	The loan title provided by the borrower
15	zip_code	The first 3 numbers of the zip code provided by the borrower in the loan application.
16	addr_state	The state provided by the borrower in the loan application
17	dti	A ratio calculated using the borrower's total monthly debt payments on the total debt obligations, excluding mortgage and the requested LC loan, divided by the borrower's self-reported monthly income.
18	earliest_cr_line	The month the borrower's earliest reported credit line was opened
19	open_acc	The number of open credit lines in the borrower's credit file.
20	pub_rec	Number of derogatory public records
21	revol_bal	Total credit revolving balance
22	revol_util	Revolving line utilization rate, or the amount of credit the borrower is using relative to all available revolving credit.
23	total_acc	The total number of credit lines currently in the borrower's credit file
24	initial_list_status	The initial listing status of the loan. Possible values are – W, F
25	application_type	Indicates whether the loan is an individual application or a joint application with two co-borrowers
26	mort_acc	Number of mortgage accounts.
27	pub_rec_bankruptcies	Number of public record bankruptcies

Type *Markdown* and LaTeX: α^2

```
In [1]: import pandas as pd
```

```
In [2]: data_info = pd.read_csv('lending_club_info.csv', index_col='LoanStatNew')
```

```
In [3]: print(data_info.loc['revol_util']['Description'])
```

Revolving line utilization rate, or the amount of credit the borrower is using relative to all available revolving credit.

```
In [4]: def feat_info(col_name):  
        print(data_info.loc[col_name]['Description'])
```

```
In [5]: feat_info('mort_acc')
```

Number of mortgage accounts.

Loading the data and other imports

```
In [6]: import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# might be needed depending on your version of Jupyter  
%matplotlib inline
```

```
In [7]: df = pd.read_csv('lending_club_loan_two.csv')
```

In [14]: df.describe

```
Out[14]: <bound method NDFrame.describe of
stallment grade sub_grade \
0      10000.0  36 months  11.44      329.48      B      B4
1       8000.0  36 months  11.99      265.68      B      B5
2      15600.0  36 months  10.49      506.97      B      B3
3       7200.0  36 months   6.49      220.65      A      A2
4      24375.0  60 months  17.27      609.33      C      C5
...      ...      ...      ...      ...      ...      ...
396025  10000.0  60 months  10.99      217.38      B      B4
396026  21000.0  36 months  12.29      700.42      C      C1
396027   5000.0  36 months   9.99      161.32      B      B1
396028  21000.0  60 months  15.31      503.02      C      C2
396029   2000.0  36 months  13.61       67.98      C      C2

      emp_title emp_length home_ownership  annual_inc  ...
\
0      Marketing  10+ years      RENT  117000.0  ...
1    Credit analyst   4 years  MORTGAGE  65000.0  ...
2    Statistician  < 1 year      RENT  43057.0  ...
3    Client Advocate   6 years      RENT  54000.0  ...
4  Destiny Management Inc.   9 years  MORTGAGE  55000.0  ...
...      ...      ...      ...      ...      ...
396025  licensed bankere   2 years      RENT  40000.0  ...
396026      Agent   5 years  MORTGAGE  110000.0  ...
396027  City Carrier  10+ years      RENT  56500.0  ...
396028  Gracon Services, Inc  10+ years  MORTGAGE  64000.0  ...
396029  Internal Revenue Service  10+ years      RENT  42996.0  ...

      open_acc pub_rec revol_bal revol_util total_acc  initial_list_status
\
0      16.0    0.0  36369.0    41.8    25.0
1      17.0    0.0  20131.0    53.3    27.0
2      13.0    0.0  11987.0    92.2    26.0
3       6.0    0.0   5472.0    21.5    13.0
4      13.0    0.0  24584.0    69.8    43.0
...      ...      ...      ...      ...      ...
396025   6.0    0.0   1990.0    34.3    23.0
396026   6.0    0.0  43263.0    95.7     8.0
396027  15.0    0.0  32704.0    66.9    23.0
396028   9.0    0.0  15704.0    53.8    20.0
396029   3.0    0.0   4292.0    91.3    19.0

      application_type  mort_acc  pub_rec_bankruptcies  \
0      INDIVIDUAL      0.0      0.0
1      INDIVIDUAL      3.0      0.0
2      INDIVIDUAL      0.0      0.0
3      INDIVIDUAL      0.0      0.0
4      INDIVIDUAL      1.0      0.0
...      ...      ...      ...
396025  INDIVIDUAL      0.0      0.0
396026  INDIVIDUAL      1.0      0.0
396027  INDIVIDUAL      0.0      0.0
396028  INDIVIDUAL      5.0      0.0
396029  INDIVIDUAL      NaN      0.0
```

```

                                address
0      0174 Michelle Gateway\nMendozaberg, OK 22690
1      1076 Carney Fort Apt. 347\nLoganmouth, SD 05113
2      87025 Mark Dale Apt. 269\nNew Sabrina, WV 05113
3      823 Reid Ford\nDelacruzside, MA 00813
4      679 Luna Roads\nGreggshire, VA 11650
...
396025 12951 Williams Crossing\nJohnnyville, DC 30723
396026 0114 Fowler Field Suite 028\nRachelborough, LA...
396027 953 Matthew Points Suite 414\nReedfort, NY 70466
396028 7843 Blake Freeway Apt. 229\nNew Michael, FL 2...
396029 787 Michelle Causeway\nBriannaton, AR 48052

[396030 rows x 27 columns]>

```

In [8]: `df.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 396030 entries, 0 to 396029
Data columns (total 27 columns):
loan_amnt      396030 non-null float64
term           396030 non-null object
int_rate       396030 non-null float64
installment    396030 non-null float64
grade          396030 non-null object
sub_grade      396030 non-null object
emp_title      373103 non-null object
emp_length     377729 non-null object
home_ownership 396030 non-null object
annual_inc     396030 non-null float64
verification_status 396030 non-null object
issue_d        396030 non-null object
loan_status    396030 non-null object
purpose        396030 non-null object
title          394275 non-null object
dti            396030 non-null float64
earliest_cr_line 396030 non-null object
open_acc       396030 non-null float64
pub_rec        396030 non-null float64
revol_bal      396030 non-null float64
revol_util     395754 non-null float64
total_acc      396030 non-null float64
initial_list_status 396030 non-null object
application_type 396030 non-null object
mort_acc       358235 non-null float64
pub_rec_bankruptcies 395495 non-null float64
address        396030 non-null object
dtypes: float64(12), object(15)
memory usage: 81.6+ MB

```

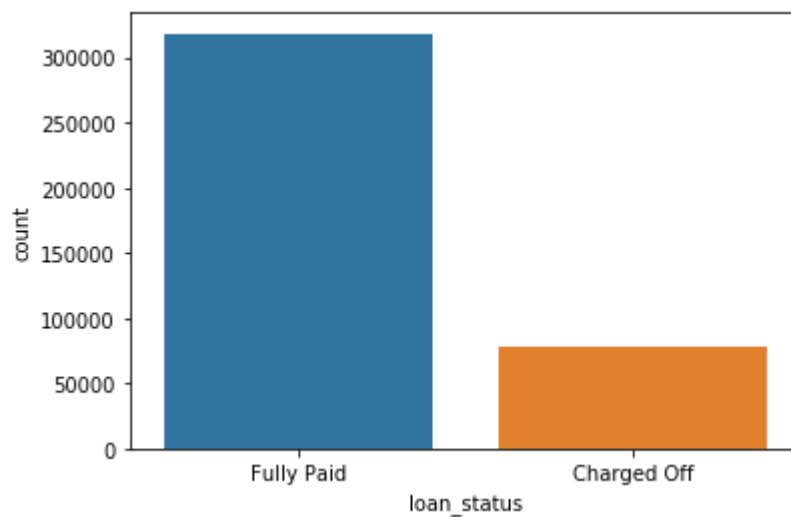
Exploratory Data Analysis

Understanding for which variables are important, view summary statistics, and visualize the data

Type *Markdown* and LaTeX: α^2

```
In [9]: sns.countplot(data = df,x = 'loan_status')
```

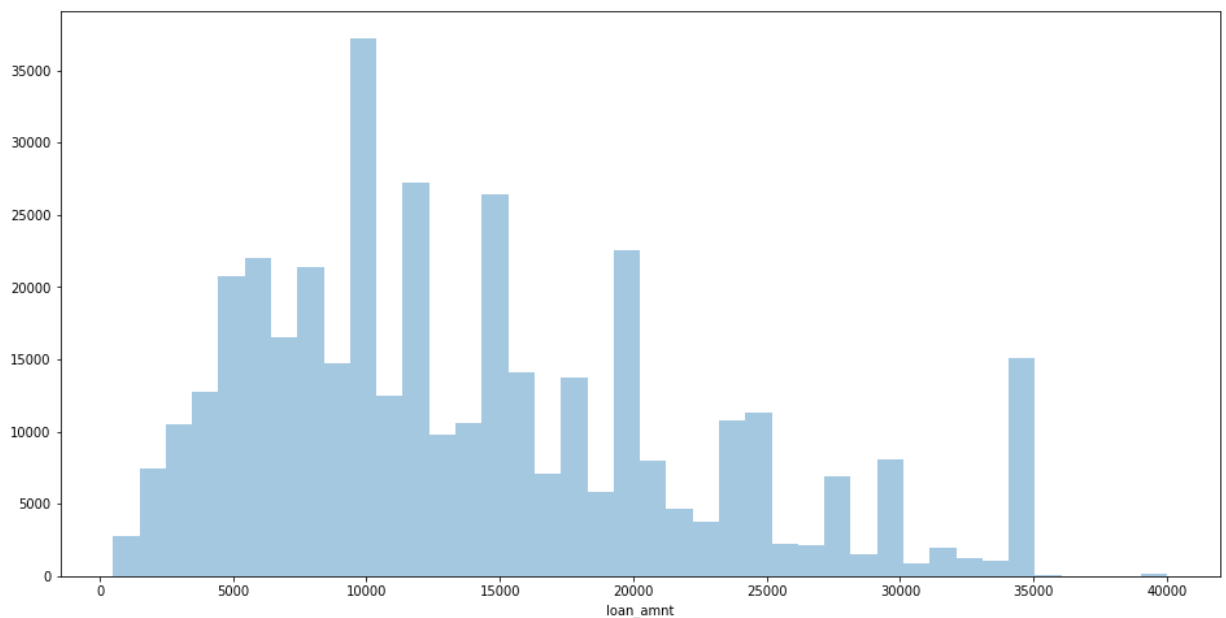
```
Out[9]: <matplotlib.axes._subplots.AxesSubplot at 0x1673ddb1188>
```



Type *Markdown* and LaTeX: α^2

```
In [10]: plt.figure(figsize=(16,8))
sns.distplot(a = df['loan_amnt'],kde = False, bins= 40)
```

```
Out[10]: <matplotlib.axes._subplots.AxesSubplot at 0x1673673fd48>
```



Type *Markdown* and LaTeX: α^2

In [11]: `df.corr()`

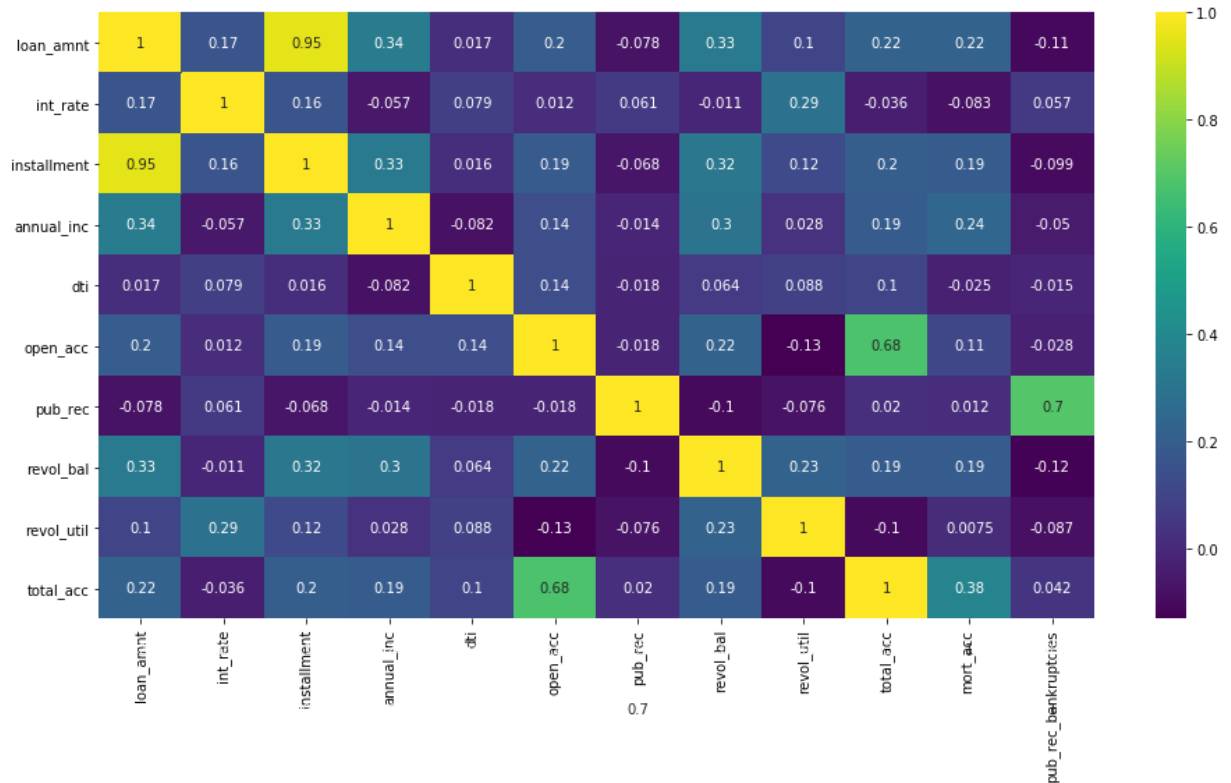
Out[11]:

	loan_amnt	int_rate	installment	annual_inc	dti	open_acc	pub_rec
loan_amnt	1.000000	0.168921	0.953929	0.336887	0.016636	0.198556	-0.077779
int_rate	0.168921	1.000000	0.162758	-0.056771	0.079038	0.011649	0.060986
installment	0.953929	0.162758	1.000000	0.330381	0.015786	0.188973	-0.067892
annual_inc	0.336887	-0.056771	0.330381	1.000000	-0.081685	0.136150	-0.013720
dti	0.016636	0.079038	0.015786	-0.081685	1.000000	0.136181	-0.017639
open_acc	0.198556	0.011649	0.188973	0.136150	0.136181	1.000000	-0.018392
pub_rec	-0.077779	0.060986	-0.067892	-0.013720	-0.017639	-0.018392	1.000000
revol_bal	0.328320	-0.011280	0.316455	0.299773	0.063571	0.221192	-0.101664
revol_util	0.099911	0.293659	0.123915	0.027871	0.088375	-0.131420	-0.075910
total_acc	0.223886	-0.036404	0.202430	0.193023	0.102128	0.680728	0.019723
mort_acc	0.222315	-0.082583	0.193694	0.236320	-0.025439	0.109205	0.011552
pub_rec_bankruptcies	-0.106539	0.057450	-0.098628	-0.050162	-0.014558	-0.027732	0.699408

Type *Markdown* and LaTeX: α^2

```
In [12]: plt.figure(figsize=(16,8))
sns.heatmap(data= df.corr(),annot = True,cmap = 'viridis')
plt.ylim(10,0)
```

Out[12]: (10, 0)



Type Markdown and LaTeX: α^2

```
In [13]: feat_info('installment')
```

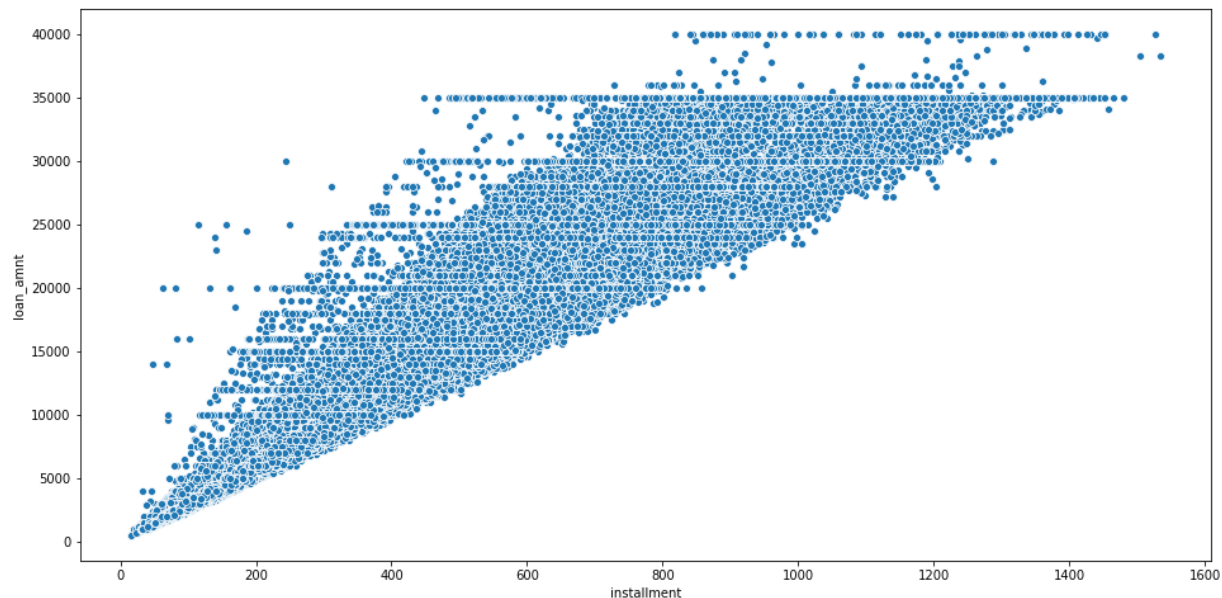
The monthly payment owed by the borrower if the loan originates.

```
In [14]: feat_info('loan_amnt')
```

The listed amount of the loan applied for by the borrower. If at some point in time, the credit department reduces the loan amount, then it will be reflected in this value.

```
In [15]: plt.figure(figsize=(16,8))  
sns.scatterplot(x = df['installment'], y= df['loan_amnt'])
```

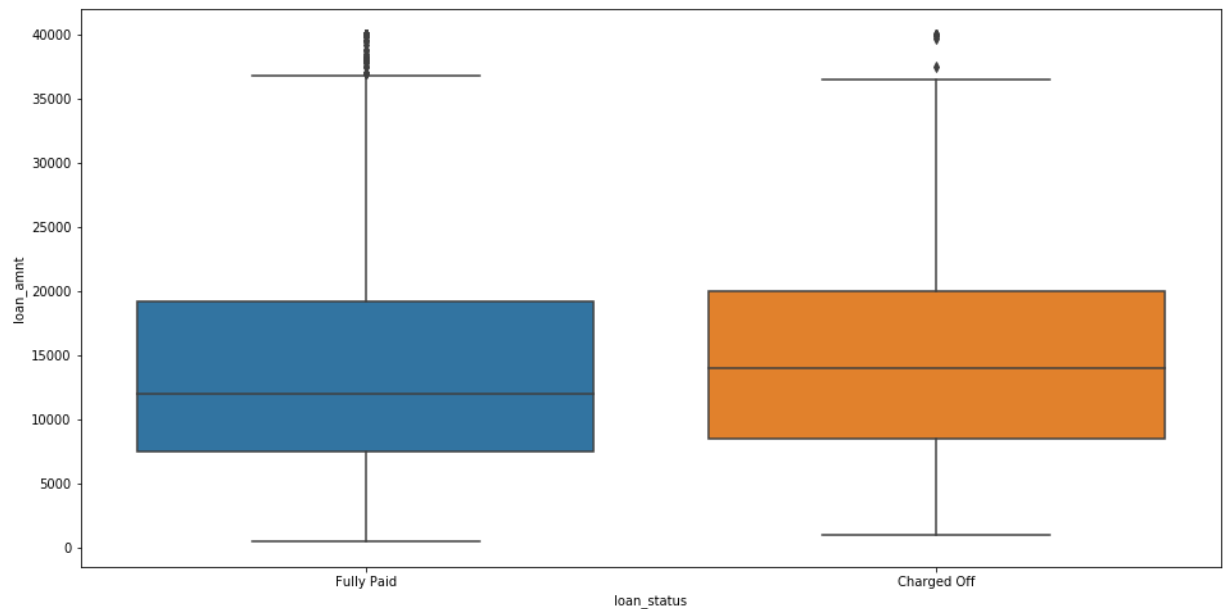
Out[15]: <matplotlib.axes._subplots.AxesSubplot at 0x16738940608>



Type *Markdown* and LaTeX: α^2

```
In [16]: plt.figure(figsize=(16,8))
sns.boxplot(x = df['loan_status'], y = df['loan_amnt'])
```

Out[16]: <matplotlib.axes._subplots.AxesSubplot at 0x167389a3748>



Type *Markdown* and LaTeX: α^2

```
In [17]: df.groupby('loan_status')['loan_amnt'].describe()
```

Out[17]:

	count	mean	std	min	25%	50%	75%	max
loan_status								
Charged Off	77673.0	15126.300967	8505.090557	1000.0	8525.0	14000.0	20000.0	40000.0
Fully Paid	318357.0	13866.878771	8302.319699	500.0	7500.0	12000.0	19225.0	40000.0

Type *Markdown* and LaTeX: α^2

```
In [18]: df['grade'].unique()
```

```
Out[18]: array(['B', 'A', 'C', 'E', 'D', 'F', 'G'], dtype=object)
```

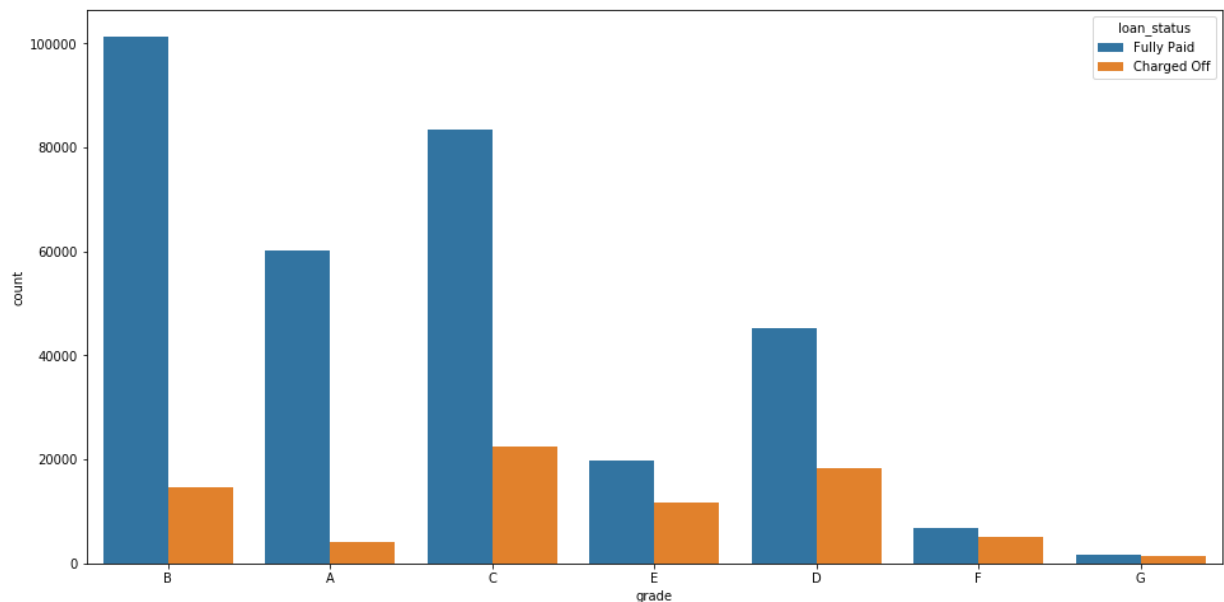
```
In [19]: df['sub_grade'].unique()
```

```
Out[19]: array(['B4', 'B5', 'B3', 'A2', 'C5', 'C3', 'A1', 'B2', 'C1', 'A5', 'E4',  
                'A4', 'A3', 'D1', 'C2', 'B1', 'D3', 'D5', 'D2', 'E1', 'E2', 'E5',  
                'F4', 'E3', 'D4', 'G1', 'F5', 'G2', 'C4', 'F1', 'F3', 'G5', 'G4',  
                'F2', 'G3'], dtype=object)
```

Type *Markdown* and LaTeX: α^2

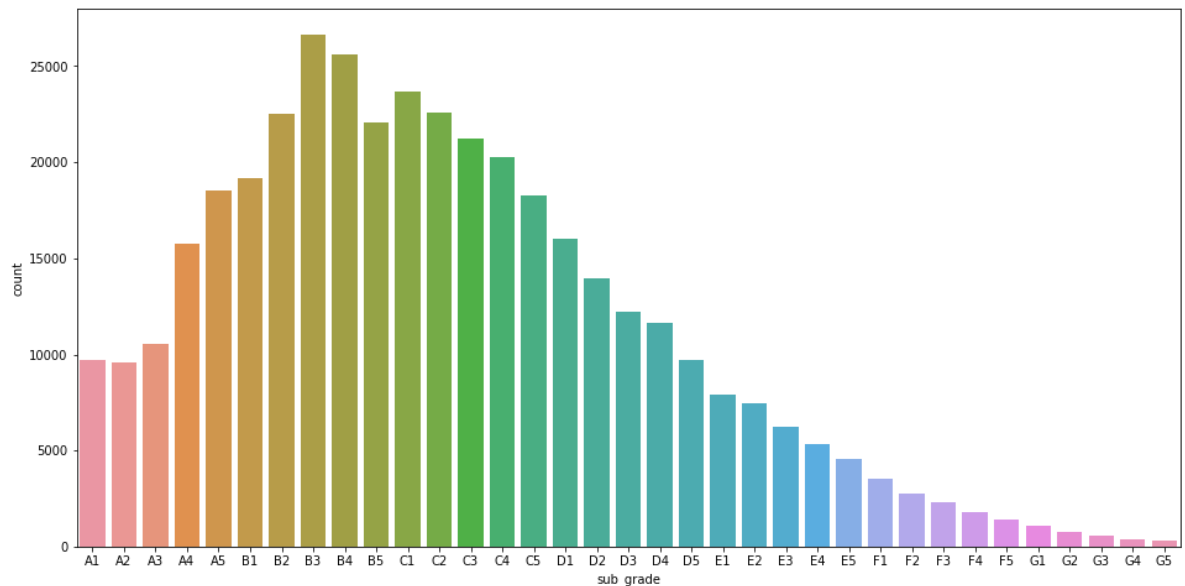
```
In [20]: plt.figure(figsize=(16,8))  
sns.countplot(x = 'grade',data=df,hue = 'loan_status')
```

```
Out[20]: <matplotlib.axes._subplots.AxesSubplot at 0x16739b5be08>
```



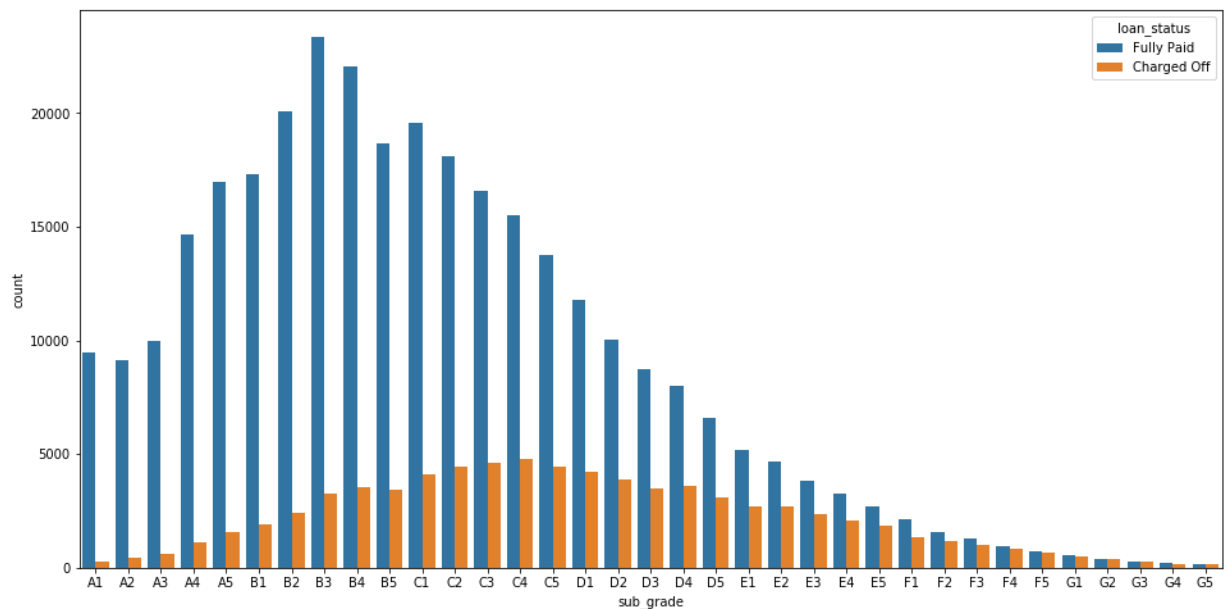
```
In [21]: plt.figure(figsize=(16,8))
sns.countplot(x = 'sub_grade', data=df, order = sorted(df['sub_grade'].unique()))
```

Out[21]: <matplotlib.axes._subplots.AxesSubplot at 0x167368f8ec8>



```
In [22]: plt.figure(figsize=(16,8))
sns.countplot(x = 'sub_grade', data=df, order = sorted(df['sub_grade'].unique()))
```

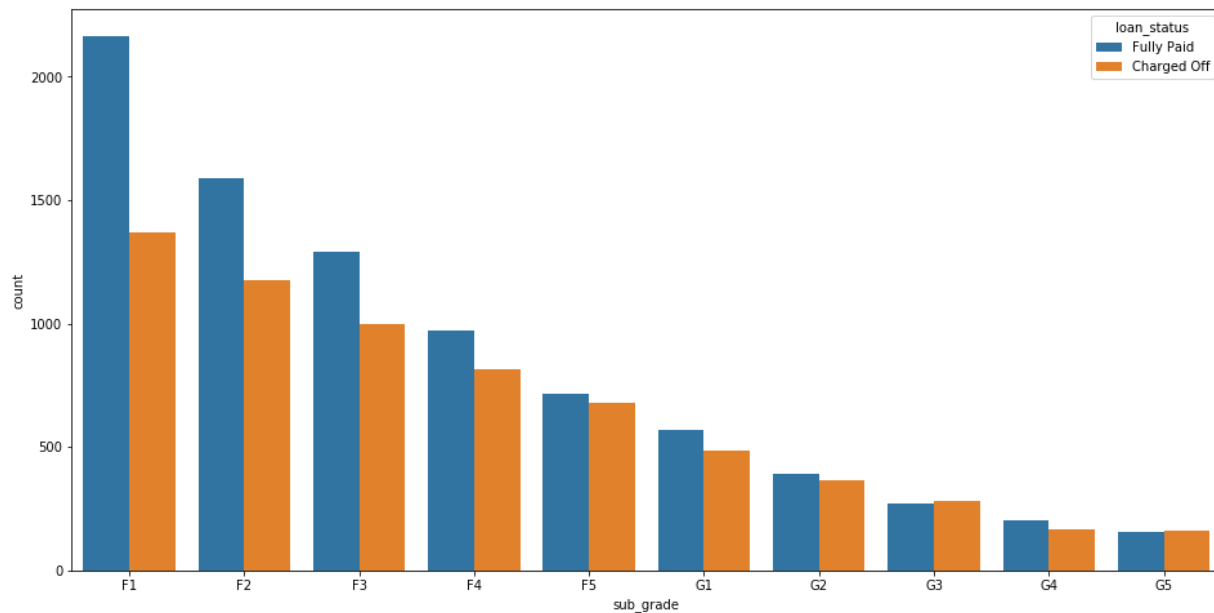
Out[22]: <matplotlib.axes._subplots.AxesSubplot at 0x167388df248>



```
In [23]: fg = df[(df['grade'] == 'G') | (df['grade'] == 'F')]
```

```
In [24]: plt.figure(figsize=(16,8))  
sns.countplot(x = 'sub_grade', data=fg, order = sorted(fg['sub_grade'].unique()))
```

Out[24]: <matplotlib.axes._subplots.AxesSubplot at 0x1673691ba88>



```
In [25]: df['loan_repaid'] = df['loan_status'].map({'Fully Paid':1,'Charged Off': 0})
```

```
In [26]: df[['loan_status', 'loan_repaid']]
```

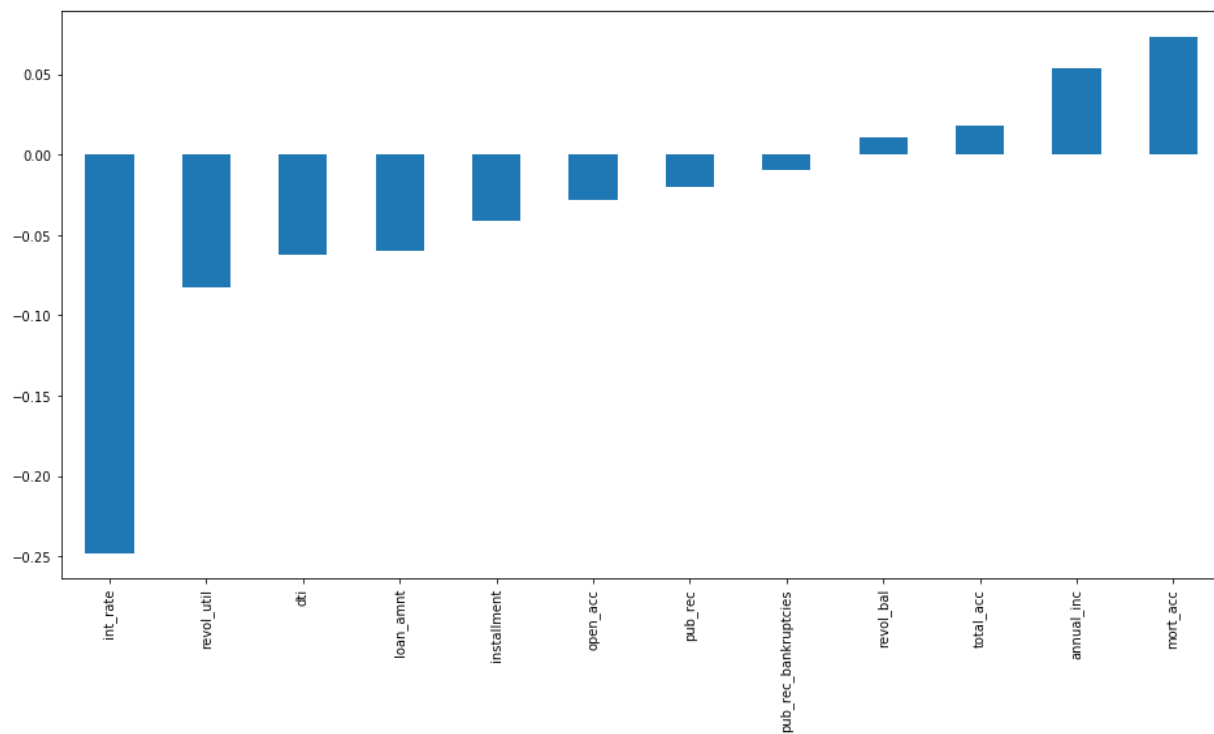
```
Out[26]:
```

	loan_status	loan_repaid
0	Fully Paid	1
1	Fully Paid	1
2	Fully Paid	1
3	Fully Paid	1
4	Charged Off	0
...	...	...
396025	Fully Paid	1
396026	Fully Paid	1
396027	Fully Paid	1
396028	Fully Paid	1
396029	Fully Paid	1

396030 rows × 2 columns

```
In [27]: plt.figure(figsize=(16,8))  
df.corr()['loan_repaid'].sort_values().drop('loan_repaid').plot(kind='bar')
```

```
Out[27]: <matplotlib.axes._subplots.AxesSubplot at 0x1673a35a208>
```

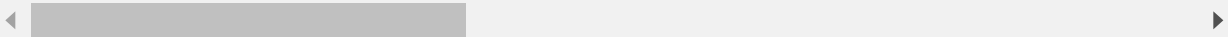


```
In [28]: df.head()
```

```
Out[28]:
```

	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	home_ow
0	10000.0	36 months	11.44	329.48	B	B4	Marketing	10+ years	
1	8000.0	36 months	11.99	265.68	B	B5	Credit analyst	4 years	MOR
2	15600.0	36 months	10.49	506.97	B	B3	Statistician	< 1 year	
3	7200.0	36 months	6.49	220.65	A	A2	Client Advocate	6 years	
4	24375.0	60 months	17.27	609.33	C	C5	Destiny Management Inc.	9 years	MOR

5 rows × 28 columns



Missing Data

Exploring missing data columns.I used a variety of factors to decide whether or not they would be useful, to see if we should keep, discard, or fill in the missing data.

```
In [29]: len(df)
```

```
Out[29]: 396030
```

```
In [30]: df.isnull().sum()
```

```
Out[30]: loan_amnt      0
term      0
int_rate  0
installment  0
grade     0
sub_grade 0
emp_title 22927
emp_length 18301
home_ownership  0
annual_inc  0
verification_status  0
issue_d      0
loan_status  0
purpose     0
title      1755
dti         0
earliest_cr_line  0
open_acc    0
pub_rec     0
revol_bal   0
revol_util  276
total_acc   0
initial_list_status  0
application_type  0
mort_acc    37795
pub_rec_bankruptcies  535
address     0
loan_repaid  0
dtype: int64
```

```
In [31]: df.isnull().sum() / len(df) * 100
```

```
Out[31]: loan_amnt      0.000000
term      0.000000
int_rate  0.000000
installment 0.000000
grade     0.000000
sub_grade 0.000000
emp_title  5.789208
emp_length 4.621115
home_ownership 0.000000
annual_inc 0.000000
verification_status 0.000000
issue_d    0.000000
loan_status 0.000000
purpose    0.000000
title      0.443148
dti        0.000000
earliest_cr_line 0.000000
open_acc   0.000000
pub_rec    0.000000
revol_bal  0.000000
revol_util 0.069692
total_acc  0.000000
initial_list_status 0.000000
application_type 0.000000
mort_acc   9.543469
pub_rec_bankruptcies 0.135091
address     0.000000
loan_repaid 0.000000
dtype: float64
```

```
In [32]: feat_info('emp_title')
```

The job title supplied by the Borrower when applying for the loan.*

```
In [33]: df['emp_title'].nunique()
```

```
Out[33]: 173105
```

```
In [34]: df['emp_title'].value_counts()
```

```
Out[34]: Teacher      4389
Manager      4250
Registered Nurse  1856
RN           1846
Supervisor   1830
...
Bonillas Elementary      1
mattress professional    1
Valley Plating           1
Enteractive Solutions Group, Inc 1
Deputy Commander        1
Name: emp_title, Length: 173105, dtype: int64
```

```
In [35]: df = df.drop('emp_title',axis = 1)
```

```
In [36]: sorted(df['emp_length'].dropna().unique())
```

```
Out[36]: ['1 year',  
          '10+ years',  
          '2 years',  
          '3 years',  
          '4 years',  
          '5 years',  
          '6 years',  
          '7 years',  
          '8 years',  
          '9 years',  
          '< 1 year']
```

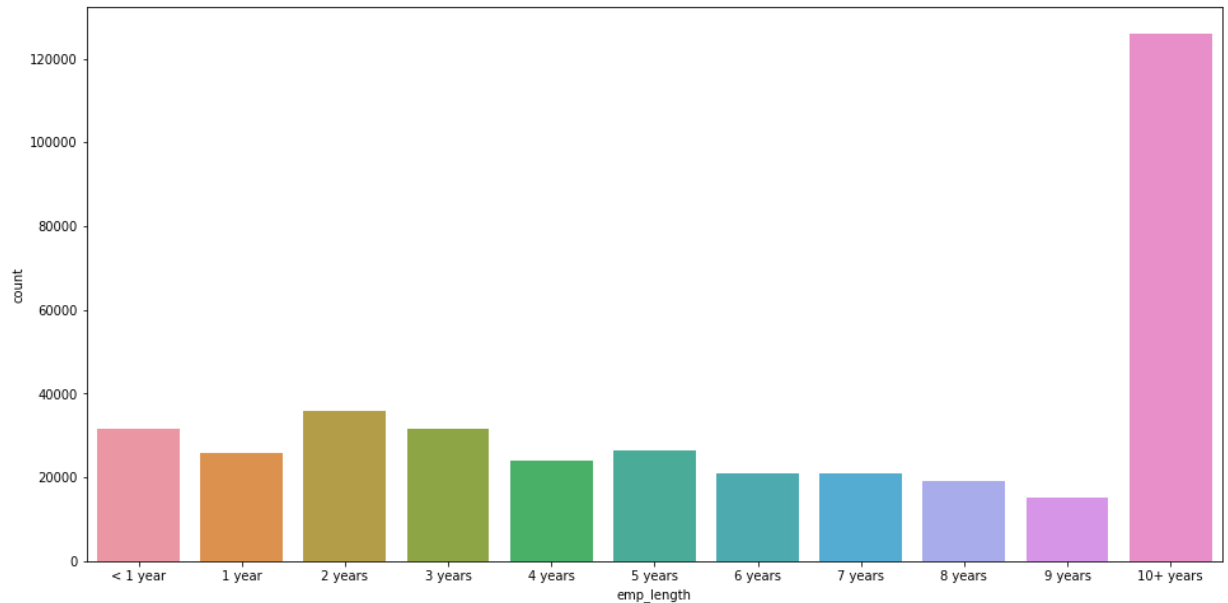
```
In [37]: emp_length_order = ['< 1 year', '1 year', '2 years', '3 years', '4 years', '5 years',  
                             '10+ years']
```

```
In [38]: emp_length_order
```

```
Out[38]: ['< 1 year',  
          '1 year',  
          '2 years',  
          '3 years',  
          '4 years',  
          '5 years',  
          '6 years',  
          '7 years',  
          '8 years',  
          '9 years',  
          '10+ years']
```

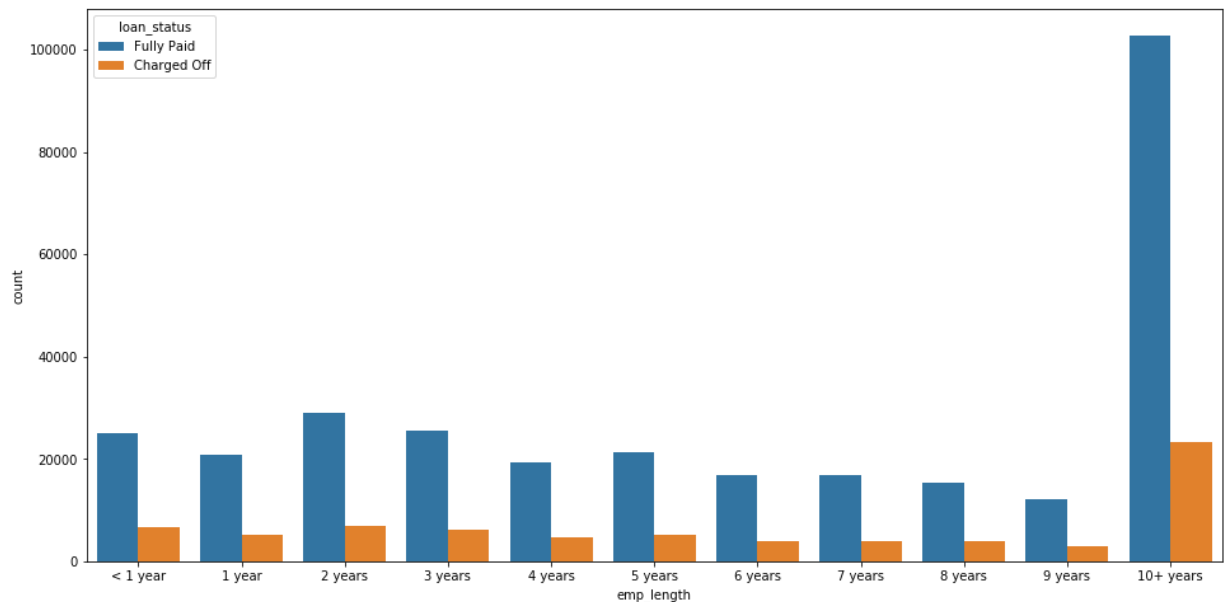
```
In [39]: plt.figure(figsize=(16,8))
sns.countplot(x = 'emp_length',data = df, order = emp_length_order)
```

Out[39]: <matplotlib.axes._subplots.AxesSubplot at 0x1673994e7c8>



```
In [40]: plt.figure(figsize=(16,8))
sns.countplot(x = 'emp_length',data = df, order = emp_length_order, hue = 'loan_status')
```

Out[40]: <matplotlib.axes._subplots.AxesSubplot at 0x1673a7c7588>



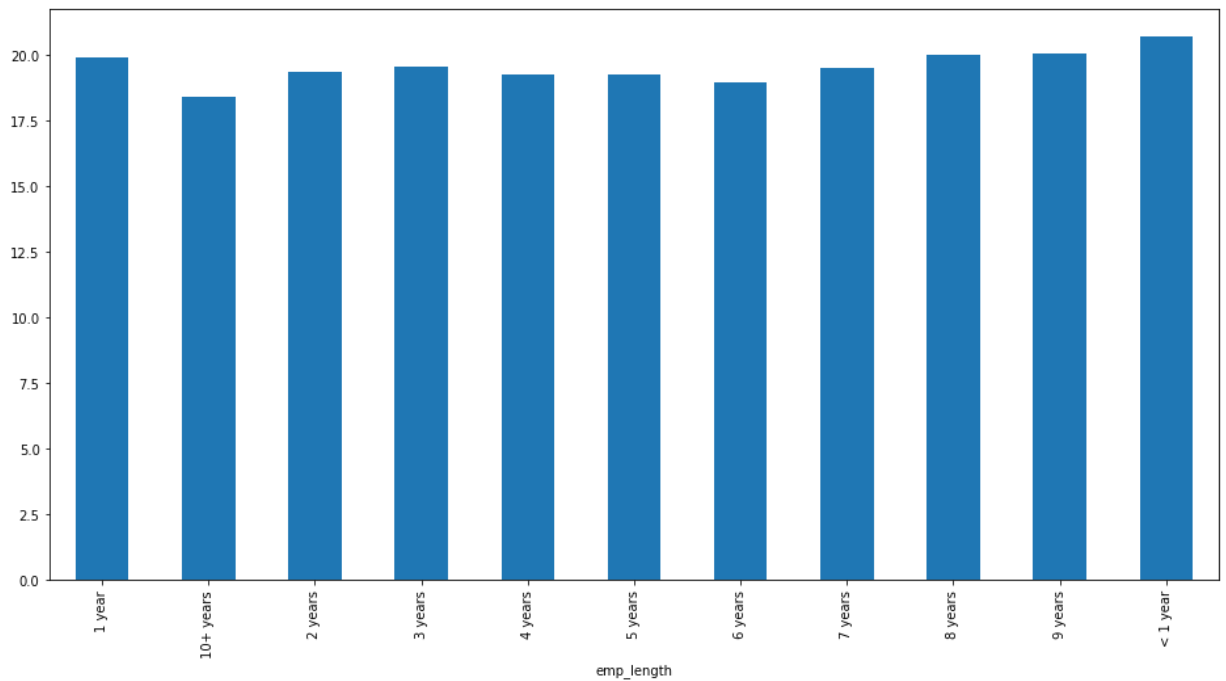
```
In [41]: emp_co = df[df['loan_status']=='Charged Off'].groupby('emp_length').count()['loan_status']
```

```
In [42]: emp_fp = df[df['loan_status']=='Fully Paid'].groupby('emp_length').count()['loan_status']
```

```
In [43]: emp_len = emp_co/(emp_fp+emp_co)*100
```

```
In [44]: plt.figure(figsize=(16,8))  
emp_len.plot(kind = 'bar')
```

```
Out[44]: <matplotlib.axes._subplots.AxesSubplot at 0x16744c54f08>
```



```
In [45]: df = df.drop('emp_length',axis = 1)
```

```
In [46]: df.isnull().sum()
```

```
Out[46]: loan_amnt      0
term      0
int_rate  0
installment  0
grade     0
sub_grade  0
home_ownership  0
annual_inc  0
verification_status  0
issue_d    0
loan_status  0
purpose    0
title      1755
dti        0
earliest_cr_line  0
open_acc   0
pub_rec    0
revol_bal  0
revol_util  276
total_acc  0
initial_list_status  0
application_type  0
mort_acc   37795
pub_rec_bankruptcies  535
address     0
loan_repaid  0
dtype: int64
```

```
In [47]: df['purpose'].head(10)
```

```
Out[47]: 0      vacation
1  debt_consolidation
2      credit_card
3      credit_card
4      credit_card
5  debt_consolidation
6  home_improvement
7      credit_card
8  debt_consolidation
9  debt_consolidation
Name: purpose, dtype: object
```

```
In [48]: df['title'].head(10)
```

```
Out[48]: 0          Vacation
1      Debt consolidation
2  Credit card refinancing
3  Credit card refinancing
4    Credit Card Refinance
5      Debt consolidation
6      Home improvement
7    No More Credit Cards
8      Debt consolidation
9    Debt Consolidation
Name: title, dtype: object
```

```
In [49]: df = df.drop('title',axis=1)
```

```
In [50]: feat_info('mort_acc')
```

Number of mortgage accounts.

```
In [51]: df['mort_acc'].value_counts()
```

```
Out[51]: 0.0      139777
         1.0      60416
         2.0      49948
         3.0      38049
         4.0      27887
         5.0      18194
         6.0      11069
         7.0       6052
         8.0       3121
         9.0       1656
        10.0        865
        11.0        479
        12.0        264
        13.0        146
        14.0        107
        15.0         61
        16.0         37
        17.0         22
        18.0         18
        19.0         15
        20.0         13
        24.0         10
        22.0          7
        21.0          4
        25.0          4
        27.0          3
        23.0          2
        32.0          2
        26.0          2
        31.0          2
        30.0          1
        28.0          1
        34.0          1
        Name: mort_acc, dtype: int64
```

```
In [52]: df.corr()['mort_acc'].sort_values()
```

```
Out[52]: int_rate      -0.082583
         dti           -0.025439
         revol_util     0.007514
         pub_rec        0.011552
         pub_rec_bankruptcies  0.027239
         loan_repaid     0.073111
         open_acc       0.109205
         installment    0.193694
         revol_bal      0.194925
         loan_amnt      0.222315
         annual_inc     0.236320
         total_acc      0.381072
         mort_acc       1.000000
        Name: mort_acc, dtype: float64
```

```
In [53]: total_acc_avg = df.groupby('total_acc').mean()['mort_acc']
```

```
In [54]: def fill_mort_acc(total_acc,mort_acc):  
    if np.isnan(mort_acc):  
        return total_acc_avg[total_acc]  
    else:  
        return mort_acc
```

```
In [55]: df['mort_acc'] = df.apply(lambda x : fill_mort_acc(x['total_acc'],x['mort_acc'])).
```

```
In [56]: df.isnull().sum()
```

```
Out[56]: loan_amnt      0  
term      0  
int_rate  0  
installment  0  
grade      0  
sub_grade  0  
home_ownership  0  
annual_inc  0  
verification_status  0  
issue_d      0  
loan_status  0  
purpose      0  
dti      0  
earliest_cr_line  0  
open_acc      0  
pub_rec      0  
revol_bal      0  
revol_util    276  
total_acc      0  
initial_list_status  0  
application_type  0  
mort_acc      0  
pub_rec_bankruptcies    535  
address      0  
loan_repaid      0  
dtype: int64
```

```
In [57]: df = df.dropna()
```

```
In [58]: df.isnull().sum()
```

```
Out[58]: loan_amnt      0
term      0
int_rate  0
installment  0
grade     0
sub_grade 0
home_ownership  0
annual_inc  0
verification_status  0
issue_d     0
loan_status 0
purpose     0
dti         0
earliest_cr_line  0
open_acc    0
pub_rec     0
revol_bal   0
revol_util  0
total_acc   0
initial_list_status  0
application_type  0
mort_acc    0
pub_rec_bankruptcies  0
address     0
loan_repaid  0
dtype: int64
```

```
In [84]: #listing all columns which are non numeric values
df.select_dtypes(['object']).columns
```

```
Out[84]: Index(['term', 'grade', 'sub_grade', 'home_ownership', 'verification_status',
               'issue_d', 'loan_status', 'purpose', 'earliest_cr_line',
               'initial_list_status', 'application_type', 'address'],
              dtype='object')
```

```
In [86]: df['term'].value_counts()
```

```
Out[86]: 36 months    301247
        60 months    93972
        Name: term, dtype: int64
```

```
In [88]: df['term'] = df['term'].apply(lambda term: int(term[:3]))
```

```
In [90]: df['term'].value_counts()
```

```
Out[90]: 36    301247
        60    93972
        Name: term, dtype: int64
```

```
In [91]: df = df.drop('grade',axis=1)
```

```
In [92]: dummies = pd.get_dummies(df['sub_grade'],drop_first=True)
#Drop_first = true because is there A/B/C, if we drop A and B then answer is C.

df = pd.concat([df.drop('sub_grade',axis=1),dummies],axis=1)
```

```
In [94]: dummies = pd.get_dummies(df[['verification_status', 'application_type','initial_list_status']],drop_first=True)

df = pd.concat([df.drop(['verification_status', 'application_type','initial_list_status'],axis=1),dummies],axis=1)
```

```
In [96]: df.columns
```

```
Out[96]: Index(['loan_amnt', 'term', 'int_rate', 'installment', 'home_ownership',
               'annual_inc', 'issue_d', 'loan_status', 'dti', 'earliest_cr_line',
               'open_acc', 'pub_rec', 'revol_bal', 'revol_util', 'total_acc',
               'mort_acc', 'pub_rec_bankruptcies', 'address', 'loan_repaid', 'A2',
               'A3', 'A4', 'A5', 'B1', 'B2', 'B3', 'B4', 'B5', 'C1', 'C2', 'C3', 'C4',
               'C5', 'D1', 'D2', 'D3', 'D4', 'D5', 'E1', 'E2', 'E3', 'E4', 'E5', 'F1',
               'F2', 'F3', 'F4', 'F5', 'G1', 'G2', 'G3', 'G4', 'G5',
               'verification_status_Source Verified', 'verification_status_Verified',
               'application_type_INDIVIDUAL', 'application_type_JOINT',
               'initial_list_status_w', 'purpose_credit_card',
               'purpose_debt_consolidation', 'purpose_educational',
               'purpose_home_improvement', 'purpose_house', 'purpose_major_purchase',
               'purpose_medical', 'purpose_moving', 'purpose_other',
               'purpose_renewable_energy', 'purpose_small_business',
               'purpose_vacation', 'purpose_wedding'],
              dtype='object')
```

```
In [98]: df['home_ownership'].value_counts()
```

```
Out[98]: MORTGAGE      198022
          RENT          159395
          OWN           37660
          OTHER          110
          NONE           29
          ANY             3
          Name: home_ownership, dtype: int64
```

```
In [100]: df['home_ownership'] = df['home_ownership'].replace(['NONE','ANY'],'OTHER')
```

```
In [101]: df['home_ownership'].value_counts()
```

```
Out[101]: MORTGAGE      198022
           RENT          159395
           OWN           37660
           OTHER          142
           Name: home_ownership, dtype: int64
```

```
In [102]: dummies = pd.get_dummies(df['home_ownership'],drop_first=True)

df = pd.concat([df.drop('home_ownership',axis=1),dummies],axis=1)
```

```
In [105]: df['zip_code'] = df['address'].apply(lambda address: address[-5:])
```

```
In [106]: df['zip_code'].value_counts()
```

```
Out[106]: 70466    56880
          22690    56413
          30723    56402
          48052    55811
          00813    45725
          29597    45393
          05113    45300
          11650    11210
          93700    11126
          86630    10959
          Name: zip_code, dtype: int64
```

```
In [107]: dummies = pd.get_dummies(df['zip_code'], drop_first=True)

          df = pd.concat([df.drop(['zip_code'], axis=1), dummies], axis=1)
```

```
In [108]: df = df.drop('address', axis=1)
```

```
In [109]: df = df.drop('issue_d', axis = 1)
```

```
In [112]: df['earliest_cr_line'] = df['earliest_cr_line'].apply(lambda date: int(date[-4:]))
```

```
In [114]: df['earliest_cr_line'].value_counts()
```

```
Out[114]: 2000    29302
          2001    29031
          1999    26444
          2002    25849
          2003    23623
          ...
          1951     3
          1950     3
          1953     2
          1948     1
          1944     1
          Name: earliest_cr_line, Length: 65, dtype: int64
```

Train Test Split

```
In [115]: from sklearn.model_selection import train_test_split
```

```
In [116]: df = df.drop('loan_status', axis = 1)
```

```
In [117]: X = df.drop('loan_repaid', axis = 1).values
```

```
In [118]: y = df['loan_repaid'].values
```

```
In [72]: # df = df.sample(frac=0.1,random_state=101)
print(len(df))
```

```
395219
```

```
In [119]: X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random
```

Normalizing the Data

```
In [121]: from sklearn.preprocessing import MinMaxScaler
```

```
In [122]: scaler = MinMaxScaler()
```

```
In [123]: X_train = scaler.fit_transform(X_train)
```

```
In [124]: X_test = scaler.transform(X_test)
```

Creating the Model

```
In [125]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout
```

```
In [127]: model = Sequential()
model.add(Dense(78,activation = 'relu'))
model.add(Dropout(0.2))

model.add(Dense(39,activation = 'relu'))
model.add(Dropout(0.2))

model.add(Dense(19,activation = 'relu'))
model.add(Dropout(0.2))

model.add(Dense(units = 1,activation = 'sigmoid'))

model.compile (loss = 'binary_crossentropy',optimizer = 'adam')
```

```
In [128]: X_train.shape
```

```
Out[128]: (316175, 78)
```

```
In [154]: model.fit(x = X_train,y = y_train,epochs = 200, batch_size = 256, validation_data = (X_val, y_val))
```

Train on 316175 samples, validate on 79044 samples

Epoch 1/200

316175/316175 [=====] - 8s 27us/sample - loss: 0.255

9 - val_loss: 0.2610

Epoch 2/200

316175/316175 [=====] - 8s 26us/sample - loss: 0.255

8 - val_loss: 0.2613

Epoch 3/200

316175/316175 [=====] - 6s 19us/sample - loss: 0.255

6 - val_loss: 0.2612

Epoch 4/200

316175/316175 [=====] - 5s 14us/sample - loss: 0.255

6 - val_loss: 0.2617

Epoch 5/200

316175/316175 [=====] - 4s 14us/sample - loss: 0.255

4 - val_loss: 0.2610

Epoch 6/200

316175/316175 [=====] - 4s 11us/sample - loss: 0.255

5 - val_loss: 0.2611

Epoch 7/200

```
In [167]: from tensorflow.keras.models import load_model
```

```
In [168]: model.save('capstone_model.h5')
```

Evaluating Model Performance.

```
In [155]: losses = pd.DataFrame(model.history.history)
```

```
In [156]: losses
```

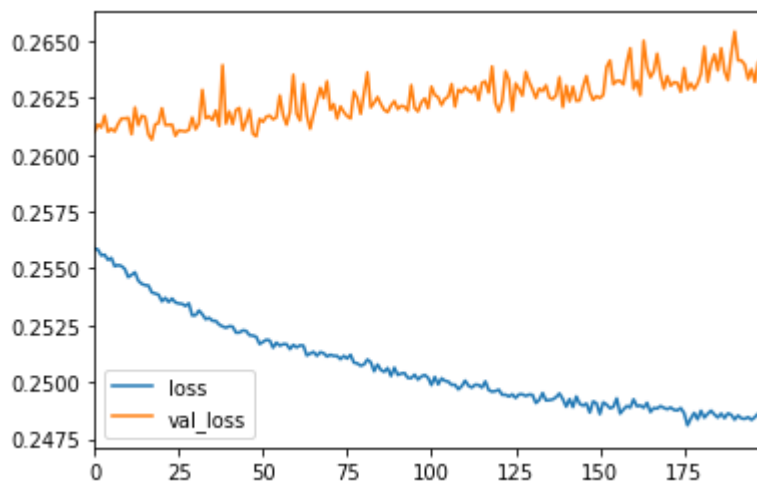
```
Out[156]:
```

	loss	val_loss
0	0.255875	0.261004
1	0.255832	0.261329
2	0.255573	0.261189
3	0.255606	0.261719
4	0.255376	0.261004
...	...	...
195	0.248347	0.263754
196	0.248464	0.263197
197	0.248622	0.264076
198	0.248569	0.263731
199	0.248004	0.263911

200 rows × 2 columns

```
In [157]: losses.plot()
```

```
Out[157]: <matplotlib.axes._subplots.AxesSubplot at 0x16701a37b88>
```



```
In [158]: from sklearn.metrics import classification_report, confusion_matrix
```

```
In [159]: predictions = model.predict_classes(X_test)
```

```
In [160]: print(classification_report(y_test,predictions))
```

	precision	recall	f1-score	support
0	0.96	0.45	0.61	15658
1	0.88	0.99	0.93	63386
accuracy			0.89	79044
macro avg	0.92	0.72	0.77	79044
weighted avg	0.90	0.89	0.87	79044

```
In [161]: df['loan_repaid'].value_counts()
```

```
Out[161]: 1    317696
          0     77523
          Name: loan_repaid, dtype: int64
```

```
In [162]: print(confusion_matrix(y_test,predictions))
```

```
[[ 7080  8578]
 [  319 63067]]
```

```
In [163]: import random
          random.seed(101)
          random_ind = random.randint(0,len(df))

          new_customer = df.drop('loan_repaid',axis=1).iloc[random_ind]
          new_customer
```

```
Out[163]: loan_amnt    25000.00
          term         60.00
          int_rate    18.24
          installment  638.11
          annual_inc   61665.00
          ...
          30723         1.00
          48052         0.00
          70466         0.00
          86630         0.00
          93700         0.00
          Name: 305323, Length: 78, dtype: float64
```

```
In [164]: new_customer = scaler.transform(new_customer.values.reshape(1,78))
```

```
In [165]: model.predict_classes(new_customer)
```

```
Out[165]: array([[1]])
```

```
In [166]: df.iloc[random_ind]['loan_repaid']
```

```
Out[166]: 1.0
```

Conclusion - The model predicted is right that the new customer will repay the loan amount

```
In [ ]:
```